DISPATCHING REQUESTS TO METHODS

Hands-On Lab Workbook

Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1 (Jersey 2.x) | Maven 3.8+
Namespace: javax.ws.rs.* (NOT jakarta.ws.rs.*)
Data format: XML — @Produces(APPLICATION_XML) / @Consumes(APPLICATION_XML)
Starter project: dispatch-lab-starter.zip
Labs: 5 Labs — approx. 3.5 hours total

© Training Materials — All Rights Reserved

Lab Overview & Prerequisites

Package Namespace & Data Format — Important

All code uses javax.ws.rs.* (NOT jakarta.ws.rs.*) — correct for Java 11 / Tomcat 9.
All resource methods use @Produces(MediaType.APPLICATION_XML) and @Consumes(MediaType.APPLICATION_XML).
Domain model classes carry @XmlRootElement so Jersey/JAXB marshals them automatically.
curl commands use -H 'Content-Type: application/xml' and -H 'Accept: application/xml'.

Labs in This Workbook

Lab	Title	Covers	Section	Time
1	ApplicationPath & Versioning	@ApplicationPath, multi-app WAR, providers	Sec 1	35 min
2	@Path Templates & Matching	Literals, variables, regex, tiebreakers	Sec 2	45 min
3	HTTP Methods & Status Codes	Verbs, XML @Produces/@Consumes, Response	Sec 3	45 min
4	Sub-resource Locators	Locator pattern, CDI, locator chains	Sec 4	40 min
5	Annotation Inheritance	Interface contracts, overriding rules	Sec 5	35 min

Starter Project Setup

1. Unzip dispatch-lab-starter.zip and import as a Maven project.
2. Run: mvn clean package -DskipTests — build must succeed.
3. Run: mvn test -Dtest=SmokeTest — all baseline tests must PASS.
4. Deploy: cp target/dispatch-lab.war $CATALINA_HOME/webapps/
5. Test: curl -H 'Accept: application/xml' http://localhost:8080/dispatch-lab/api/health
6. Expected: <?xml ...?> <health><status>UP</status>...</health>

What is pre-built

HealthResource.java — complete, returns XML <health> response
ProductRepository.java — in-memory store with 6 seed products
StoreRepository.java — in-memory store with 3 seed stores
OrderRepository.java — in-memory store, empty at start
SmokeTest.java — 10 baseline tests; must all pass before labs
model/*.java — Product, Order, Store etc. (@XmlRootElement annotated)

© Training Materials — All Rights Reserved

LAB 1

ApplicationPath & API Versioning

Section: Section 1

35 min

Objectives

- Deploy a JAX-RS application to Tomcat 9 and verify @ApplicationPath routing
- Confirm that the three-part URI (context root + app path + resource path) resolves correctly
- Deploy a second Application at /api/v2 in the same WAR
- Observe that providers registered in one Application are not visible in another

Package Namespace & Data Format — Important

All code uses javax.ws.rs.* (NOT jakarta.ws.rs.*) — correct for Java 11 / Tomcat 9.
All resource methods use @Produces(MediaType.APPLICATION_XML) and @Consumes(MediaType.APPLICATION_XML).
Domain model classes carry @XmlRootElement so Jersey/JAXB marshals them automatically.
curl commands use -H 'Content-Type: application/xml' and -H 'Accept: application/xml'.

Task 1.1 — Baseline Verification

Build and deploy the starter WAR. Verify the pre-built health endpoint responds with XML:

```
mvn clean package -DskipTests
cp target/dispatch-lab.war $CATALINA_HOME/webapps/
$CATALINA_HOME/bin/startup.sh

curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/health

# Expected XML response:
# <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
# <health>
#   <status>UP</status>
#   <service>DispatchLab</service>
#   <version>1.0</version>
#   <timestamp>2024-06-15T10:30:00</timestamp>
# </health>
```

Paste the XML health response here:

Task 1.2 — Complete ShopApplication

Open ShopApplication.java. Add getClasses() to register components manually, using javax.ws.rs.core.Application:

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;
```

© Training Materials — All Rights Reserved

```
import java.util.Set;

@ApplicationPath("/api")
public class ShopApplication extends Application {
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(
            HealthResource.class
            // TODO: add ProductResource here in Lab 2
        );
    }
}

# After redeployment — test both endpoints:
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/health
# → 200 XML

curl -v http://localhost:8080/dispatch-lab/api/products
# → 404 (not yet registered)
```

Q1

What HTTP status code does /api/products return after you switch to manual registration? Why? What is the JAX-RS rule that explains this behaviour?

Your answer:

Task 1.3 — Create a Second Application (API v2)

Create ApiV2Application.java mapped to /api/v2. Register a V2HealthResource that returns a version-2 XML response:

```
// V2HealthResource.java
import javax.ws.rs.*;
import javax.ws.rs.core.MediaType;
import javax.xml.bind.annotation.*;

@XmlRootElement(name = "health")
@XmlAccessorType(XmlAccessType.FIELD)
class V2HealthResponse {
    @XmlElement public String status      = "UP";
    @XmlElement public int    apiVersion  = 2;
    @XmlElement public String message     = "Welcome to API v2";
    public V2HealthResponse() {}
}

@Path("/health")
public class V2HealthResource {
    @GET
    @Produces(MediaType.APPLICATION_XML)
    public V2HealthResponse check() {
        return new V2HealthResponse();
    }
```

© Training Materials — All Rights Reserved

```
}

// ApiV2Application.java
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api/v2")
public class ApiV2Application extends Application {
    @Override public Set<Class<?>> getClasses() {
        return Set.of(V2HealthResource.class);
    }
}

# Test both versions:
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/health
# → <health><status>UP</status>...</health>

curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/v2/health
# → <health><status>UP</status><apiVersion>2</apiVersion>...</health>
```

Q2

If you register a CORS ContainerResponseFilter in ShopApplication, does it run for requests to /api/v2/health? Test it. What does this tell you about provider visibility across Application instances?

Your findings:

Lab 1 Checkpoint

- ✓ GET /api/health returns XML <health><status>UP</status>...</health>
- ✓ GET /api/v2/health returns XML with <apiVersion>2</apiVersion>
- ✓ GET /api/products returns 404 (not yet registered in ShopApplication)
- ✓ Providers in ShopApplication are NOT visible in ApiV2Application

© Training Materials — All Rights Reserved

LAB 2

@Path Templates — Literals, Variables and Regex

Section: Section 2

45 min

Objectives

- Implement a ProductResource with @Produces(APPLICATION_XML) demonstrating all @Path template forms
- Use literal sub-paths, single variables, multi-segment variables, and regex constraints
- Observe the tiebreaker rules in action by creating both literal and variable paths at the same base
- Predict which method handles each incoming request before testing

Task 2.1 — Create ProductResource Skeleton

Create ProductResource.java using javax.ws.rs.* and APPLICATION_XML throughout. Register it in ShopApplication:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import com.example.api.model.Product;
import java.util.List;

@Path("/products")
@Produces(MediaType.APPLICATION_XML)
public class ProductResource {
    private final ProductRepository repo = ProductRepository.getInstance();
    // TODO 2.2: GET /products
    // TODO 2.2: GET /products/{id}
    // TODO 2.3: GET /products/{dbId: \d+}      (numeric)
    // TODO 2.3: GET /products/{sku: [A-Z]+-\d+} (SKU code)
    // TODO 2.4: GET /products/featured          (literal)
    // TODO 2.5: GET /products/search
    // TODO 2.5: GET /products/archive/{year}/{quarter}
}
```

Task 2.2 — Basic Collection and Item Methods

```
@GET
@Produces(MediaType.APPLICATION_XML)
public List<Product> listAll(
    @QueryParam("category")               String category,
    @QueryParam("page") @DefaultValue("0") int    page,
    @QueryParam("size") @DefaultValue("20") int   size) {
    return repo.find(category, page, size, true);
    // JAXB marshals List<Product> to XML — jersey-media-jaxb handles this
}

@GET @Path("{id}")
public Response getById(@PathParam("id") String id) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();
    return Response.ok(p).build();  // JAXB marshals Product → <product .../>
}
```

© Training Materials — All Rights Reserved

```
# Test (note: Accept: application/xml header required):
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/products
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/products/P001
curl -H 'Accept: application/xml' 'http://localhost:8080/dispatch-
lab/api/products?category=books'

# Expected product XML:
# <product id="P001">
#   <n>Clean Code</n>
#   <category>books</category>
#   <price>39.99</price>
#   <stockLevel>50</stockLevel>
# </product>
```

Task 2.3 — Regex-Constrained Variables

Add two methods that match numeric IDs and SKU codes. Predict which method handles each request BEFORE running the curl:

```
// Numeric product database ID — matches /products/42
@GET @Path("{dbId: \\d+}")
public Response getByNumericId(@PathParam("dbId") int dbId) {
    Product p = repo.findByNumericId(dbId);
    if (p == null) return Response.status(404).build();
    // Return an XML wrapper indicating which method matched
    return Response.ok(p).header("X-Matched-By","numericId").build();
}

// SKU code like WIDGET-001 — matches /products/WIDGET-001
@GET @Path("{sku: [A-Z]+-\\d+}")
public Response getBySku(@PathParam("sku") String sku) {
    Product p = repo.findBySku(sku);
    if (p == null) return Response.status(404).build();
    return Response.ok(p).header("X-Matched-By","sku").build();
}
```

Predict before running — write your expected match and reason

a. GET /api/products/42
b. GET /api/products/WIDGET-001
c. GET /api/products/P001
d. GET /api/products/abc123

Your predictions, then actual results (check X-Matched-By header):

Task 2.4 — Literal Path vs Template Tiebreaker

© Training Materials — All Rights Reserved

```
@GET @Path("featured")
@Produces(MediaType.APPLICATION_XML)
public List<Product> getFeatured() {
    return repo.findAll().stream()
                .filter(Product::isFeatured)
                .collect(java.util.stream.Collectors.toList());
}

# The literal path MUST win over {id} template:
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/products/featured
# Expected: XML list of featured products, NOT a 'product with id=featured'
```

Task 2.5 — Multi-segment @PathParam

```
// GET /api/products/search?q=java&sort=price
@GET @Path("search")
public List<Product> search(
    @QueryParam("q")                          String query,
    @QueryParam("sort") @DefaultValue("name") String sort) {
    return repo.search(query, sort);
}

// GET /api/products/archive/2023/Q4
@GET @Path("archive/{year}/{quarter}")
public Response archive(
    @PathParam("year")    int    year,
    @PathParam("quarter") String quarter) {
    return Response.ok(
        new ArchiveResult(year, quarter, repo.findAll())
    ).build();
    // ArchiveResult must be @XmlRootElement annotated
}

curl -H 'Accept: application/xml' 'http://localhost:8080/dispatch-
lab/api/products/archive/2023/Q4'
curl -H 'Accept: application/xml' 'http://localhost:8080/dispatch-
lab/api/products/archive/twenty/Q1'
# What status code does the second call return? Why?
```

Q — Type conversion

When year is declared as int but 'twenty' is sent, what HTTP status does JAX-RS return and why?

Your answer:

Lab 2 Checkpoint

- ✓ GET /api/products returns XML list of all products
- ✓ GET /api/products/42 matches getByNumericId() (X-Matched-By: numericId)
- ✓ GET /api/products/WIDGET-001 matches getBySku()

© Training Materials — All Rights Reserved

✓ GET /api/products/featured returns XML featured list (NOT getById)
✓ GET /api/products/archive/twenty/Q1 returns 404 (type conversion failure)

© Training Materials — All Rights Reserved

LAB 3

HTTP Methods, Status Codes & Content Negotiation

Section: Section 3

45 min

Objectives

- Implement full CRUD on OrderResource using correct HTTP verbs and XML
- Return appropriate status codes (201, 204, 405) using the Response builder
- Add content negotiation to return XML or plain text based on Accept header
- Demonstrate 405 Method Not Allowed for unsupported verbs on a known path

Task 3.1 — OrderResource Full CRUD

Create OrderResource.java with XML as the wire format. The Order class is already @XmlRootElement annotated:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import com.example.api.model.Order;
import java.util.List;

@Path("/orders")
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public class OrderResource {
    private OrderRepository repo = OrderRepository.getInstance();

    // GET /orders  →  <orders><order>...</order></orders>
    @GET
    public List<Order> listAll() { ... }

    // GET /orders/{id}  →  <order>...</order>
    @GET @Path("{id}")
    public Response getById(@PathParam("id") String id) { ... }

    // POST /orders  body: <order>...</order>  →  201 + Location
    @POST
    public Response create(Order order, @Context UriInfo uriInfo) { ... }

    // PUT /orders/{id}  body: <order>...</order>  →  200
    @PUT @Path("{id}")
    public Response update(@PathParam("id") String id, Order order) { ... }

    // DELETE /orders/{id}  →  204 No Content
    @DELETE @Path("{id}")
    public Response delete(@PathParam("id") String id) { ... }

    // HEAD /orders/{id}  →  headers only, no body
    @HEAD @Path("{id}")
    public Response head(@PathParam("id") String id) { ... }
}
```

Write your complete OrderResource implementation:

© Training Materials — All Rights Reserved

Task 3.2 — Verify Status Codes with XML curl

```
# POST — 201 Created  (XML body)
curl -v -X POST http://localhost:8080/dispatch-lab/api/orders \
     -H 'Content-Type: application/xml' \
     -H 'Accept: application/xml' \
     -d '<?xml
version="1.0"?><order><customerId>C001</customerId><status>NEW</status></order>'
# Expected: 201 Created + Location: http://.../api/orders/ORD-001
# Body: <order id="ORD-001"><customerId>C001</customerId>...</order>

# DELETE — 204 No Content
curl -v -X DELETE http://localhost:8080/dispatch-lab/api/orders/ORD-001
# Expected: 204, no body

# 405 Method Not Allowed — PATCH not implemented
curl -v -X PATCH http://localhost:8080/dispatch-lab/api/orders/ORD-001
# Expected: 405 with Allow header listing supported methods

# 404 Not Found
curl -v -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/orders/DOES-NOT-EXIST
```

Record the status code and notable headers (Location, Allow) for each call:

Task 3.3 — Content Negotiation (XML and plain text)

Add a summary endpoint to ProductResource that negotiates between APPLICATION_XML and TEXT_PLAIN:

```
import javax.ws.rs.core.Request;
```

© Training Materials — All Rights Reserved

```
import javax.ws.rs.core.Variant;

@GET @Path("{id}/summary")
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public Response summary(@PathParam("id") String id,
                        @Context          Request req) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();

    List<Variant> variants = Variant.mediaTypes(
        MediaType.APPLICATION_XML_TYPE,
        MediaType.TEXT_PLAIN_TYPE).build();
    Variant selected = req.selectVariant(variants);
    if (selected == null) return Response.notAcceptable(variants).build();

    if (MediaType.TEXT_PLAIN_TYPE.equals(selected.getMediaType())) {
        String text = p.getId() + " | " + p.getName() + " | " + p.getPrice();
        return Response.ok(text, MediaType.TEXT_PLAIN).build();
    }
    // Default: XML — JAXB marshals Product
    return Response.ok(p).build();
}

curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/products/P001/summary
# → <product id="P001"><n>Clean Code</n>...</product>

curl -H 'Accept: text/plain' http://localhost:8080/dispatch-
lab/api/products/P001/summary
# → P001 | Clean Code | 39.99

curl -H 'Accept: text/html' http://localhost:8080/dispatch-
lab/api/products/P001/summary
# → 406 Not Acceptable
```

Q

What status code does the last curl return and which part of the dispatch algorithm produces it?

Your answer:

Lab 3 Checkpoint

- ✓ POST /api/orders returns 201 with Location header; body is XML <order>
- ✓ DELETE /api/orders/{id} returns 204
- ✓ PATCH /api/orders/{id} returns 405 with Allow header
- ✓ GET /api/products/P001/summary Accept:application/xml returns XML
- ✓ GET /api/products/P001/summary Accept:text/plain returns pipe-delimited text
- ✓ GET /api/products/P001/summary Accept:text/html returns 406

© Training Materials — All Rights Reserved

LAB 4 **Sub-resource Locators — Hierarchy Design** ⏱ 40 min

Section: Section 4

Objectives

- Implement a sub-resource locator in StoreResource that delegates to InventoryResource
- Handle the 404-guard pattern in the locator method
- Use ResourceContext to enable CDI injection in the sub-resource
- Create a locator chain (two levels of delegation)

Task 4.1 — Target URI Structure

You will implement these URIs, all returning XML:

```
GET  /api/stores                          →
<stores><store>...</store></stores>
GET  /api/stores/{storeId}                → <store>...</store>
GET  /api/stores/{storeId}/inventory      → LOCATOR →
InventoryResource.listAll()
GET  /api/stores/{storeId}/inventory/{sku} → LOCATOR →
InventoryResource.getBySku()
PUT  /api/stores/{storeId}/inventory/{sku} → LOCATOR →
InventoryResource.update()
```

Task 4.2 — Implement StoreResource with Locator

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import com.example.api.model.Store;

@Path("/stores")
@Produces(MediaType.APPLICATION_XML)
public class StoreResource {
    private StoreRepository storeRepo = StoreRepository.getInstance();

    @GET
    public List<Store> listAll() { return storeRepo.findAll(); }

    @GET @Path("{storeId}")
    public Response getById(@PathParam("storeId") String id) {
        Store s = storeRepo.findById(id);
        if (s == null) return Response.status(404).build();
        return Response.ok(s).build();
    }

    // Sub-resource LOCATOR — NO HTTP verb annotation
    @Path("{storeId}/inventory")
    public InventoryResource getInventory(
            @PathParam("storeId") String storeId) {
        // TODO: guard — throw NotFoundException if store not found
        // TODO: return new InventoryResource(store)
        throw new UnsupportedOperationException("TODO — Lab 4");
    }
}
```

© Training Materials — All Rights Reserved

Write the completed getInventory() locator method:

Task 4.3 — Implement InventoryResource

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import com.example.api.model.InventoryItem;

// No @Path at class level — path comes from the locator
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public class InventoryResource {
    private final Store store;
    public InventoryResource(Store store) { this.store = store; }

    // GET /api/stores/{storeId}/inventory
    @GET
    public List<InventoryItem> listAll() {
        // TODO: return store.getInventory()
    }

    // GET /api/stores/{storeId}/inventory/{sku}
    @GET @Path("{sku}")
    public Response getBySku(@PathParam("sku") String sku) {
        // TODO: find by sku; 404 if missing
    }

    // PUT /api/stores/{storeId}/inventory/{sku}
    // body: <inventoryItem>...</inventoryItem>
    @PUT @Path("{sku}")
    public InventoryItem update(@PathParam("sku") String sku,
                                InventoryItem item) {
        // TODO: update and return
    }
}
```

Write your completed InventoryResource class:

© Training Materials — All Rights Reserved

Task 4.4 — Test the Locator Chain

```
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/stores
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/stores/STORE-001/inventory
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/stores/STORE-001/inventory/WIDGET-001

# Test the 404 guard:
curl -v http://localhost:8080/dispatch-lab/api/stores/DOES-NOT-
EXIST/inventory
# Expected: 404

# Test PUT (XML body):
curl -v -X PUT http://localhost:8080/dispatch-lab/api/stores/STORE-
001/inventory/WIDGET-001 \
     -H 'Content-Type: application/xml' \
     -H 'Accept: application/xml' \
     -d '<?xml version="1.0"?><inventoryItem><sku>WIDGET-
001</sku><quantity>99</quantity></inventoryItem>'
```

Q1

What HTTP status does /api/stores/DOES-NOT-EXIST/inventory return? Which class throws the exception and at which step in dispatch?

Your answer:

Q2

Add @Inject to a field in InventoryResource. Deploy and call the endpoint. What error appears in the Tomcat log? Fix it using ResourceContext.initResource(). Write the corrected locator.

Your corrected locator + explanation of the error:

© Training Materials — All Rights Reserved

Lab 4 Checkpoint

- ✓ GET /api/stores returns XML <stores> list
- ✓ GET /api/stores/STORE-001/inventory returns XML inventory items
- ✓ GET /api/stores/STORE-001/inventory/WIDGET-001 returns XML item
- ✓ GET /api/stores/DOES-NOT-EXIST/inventory returns 404
- ✓ PUT /api/stores/STORE-001/inventory/WIDGET-001 with XML body updates stock

© Training Materials — All Rights Reserved

LAB 5 **Annotation Inheritance — Interface Contracts** ⏱ 35 min
Section: Section 5

Objectives

- Define a REST contract as an annotated Java interface using javax.ws.rs.*
- Implement the contract without redeclaring method-level annotations
- Deliberately break the all-or-nothing rule and observe the result
- Correctly override a method to add an additional @Produces media type

Task 5.1 — Define ProductService Interface

ProductService.java is already in the starter. Open it — it uses javax.ws.rs.* and APPLICATION_XML:

```
package com.example.api.service;

import com.example.api.model.Product;
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import java.util.List;

@Path("/products")
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public interface ProductService {

    @GET
    List<Product> listAll(
        @QueryParam("category")                String category,
        @QueryParam("page") @DefaultValue("0") int    page,
        @QueryParam("size") @DefaultValue("20") int   size);

    @GET @Path("{id}")
    Response getById(@PathParam("id") String id);

    @POST
    Response create(Product product, @Context UriInfo uriInfo);

    @PUT @Path("{id}")
    Response update(@PathParam("id") String id, Product product);

    @DELETE @Path("{id}")
    Response delete(@PathParam("id") String id);
}
```

Task 5.2 — Implement the Interface

Create ProductResourceV2.java implementing ProductService WITHOUT redeclaring method-level JAX-RS annotations:

```
// ProductResourceV2.java — implements contract from interface
public class ProductResourceV2 implements ProductService {
    private final ProductRepository repo = ProductRepository.getInstance();

    // All JAX-RS annotations are INHERITED from ProductService interface
```

© Training Materials — All Rights Reserved

```
    @Override
    public List<Product> listAll(String category, int page, int size) {
        return repo.find(category, page, size, true);
    }

    @Override
    public Response getById(String id) {
        Product p = repo.findById(id);
        return p == null
            ? Response.status(404).build()
            : Response.ok(p).build();  // JAXB marshals to XML
    }

    @Override
    public Response create(Product product, UriInfo uriInfo) {
        Product saved = repo.save(product);
        URI loc = uriInfo.getAbsolutePathBuilder()
                           .path(saved.getId()).build();
        return Response.created(loc).entity(saved).build();
    }

    @Override
    public Response update(String id, Product product) {
        Product updated = repo.update(id, product);
        return updated == null
            ? Response.status(404).build()
            : Response.ok(updated).build();
    }

    @Override
    public Response delete(String id) {
        return repo.delete(id)
            ? Response.noContent().build()
            : Response.status(404).build();
    }
}

# Register in ApiV2Application and test:
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/v2/products
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/v2/products/P001
```

Q1

Did you need to add @Path("/products") to ProductResourceV2? Where is it declared and how does the implementation class know about it?

Your answer:

Task 5.3 — Break the All-or-Nothing Rule (Deliberately)

Add this INCORRECT method override to ProductResourceV2 — only @Produces is redeclared, @GET is missing:

```
// WRONG — @GET is missing. Only @Produces is redeclared.
```

© Training Materials — All Rights Reserved

```
@Override
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public List<Product> listAll(String category, int page, int size) {
    return repo.findAll();
}

# Deploy and test:
curl -v -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/v2/products
# What status code? Why?

# Fix — redeclare ALL parent annotations:
@Override
@GET                // MUST redeclare
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public List<Product> listAll(String category, int page, int size) {
    return repo.findAll();
}

# Re-test both Accept types:
curl -H 'Accept: application/xml' http://localhost:8080/dispatch-
lab/api/v2/products
# → XML list

curl -H 'Accept: text/plain' http://localhost:8080/dispatch-
lab/api/v2/products
# → 406 (List<Product> has no text/plain writer — implement one or return
count)
```

What status code does the broken override return? Explain the JAX-RS all-or-nothing inheritance rule:

Task 5.4 — Design Question

Architecture question

A colleague proposes putting ALL JAX-RS annotations (including @GET, @Path etc.) directly on the implementing class rather than on the interface, arguing it gives more flexibility. Evaluate this proposal:

a) What advantage does the interface-annotation approach give for testability?
b) What happens if you have two implementations of the same interface both registered in the same Application?
c) How does MicroProfile REST Client use an annotated interface?

Your evaluation:

© Training Materials — All Rights Reserved

Lab 5 Checkpoint

- ✓ GET /api/v2/products dispatches to ProductResourceV2.listAll() returning XML
- ✓ No JAX-RS annotations on implementing class methods (except where overriding)
- ✓ Broken partial override makes GET /api/v2/products return 404 or 405
- ✓ Corrected full override (with @GET) returns XML products list

© Training Materials — All Rights Reserved

Model Answers — Key Code

Reference implementations. Consult only after attempting each lab.

Lab 1 — ApiV2Application

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api/v2")
public class ApiV2Application extends Application {
    @Override public Set<Class<?>> getClasses() {
        return Set.of(V2HealthResource.class, ProductResourceV2.class);
    }
}
```

Lab 2 — Regex path methods

```
@GET @Path("{dbId: \\d+}")
@Produces(MediaType.APPLICATION_XML)
public Response getByNumericId(@PathParam("dbId") int dbId) {
    Product p = repo.findByNumericId(dbId);
    if (p == null) return Response.status(404).build();
    return Response.ok(p).header("X-Matched-By","numericId").build();
}

@GET @Path("{sku: [A-Z]+-\\d+}")
@Produces(MediaType.APPLICATION_XML)
public Response getBySku(@PathParam("sku") String sku) {
    Product p = repo.findBySku(sku);
    if (p == null) return Response.status(404).build();
    return Response.ok(p).header("X-Matched-By","sku").build();
}
```

Lab 3 — OrderResource create()

```
@POST
@Consumes(MediaType.APPLICATION_XML)
public Response create(Order order, @Context UriInfo uriInfo) {
    Order saved = repo.save(order);
    URI location = uriInfo.getAbsolutePathBuilder()
                          .path(saved.getId()).build();
    return Response.created(location).entity(saved).build();
    // Body: JAXB marshals saved Order → XML; status 201
}
```

Lab 4 — StoreResource locator

```
@Path("{storeId}/inventory")
public InventoryResource getInventory(
        @PathParam("storeId") String storeId) {
```

© Training Materials — All Rights Reserved

```
    Store store = storeRepo.findById(storeId);
    if (store == null)
        throw new javax.ws.rs.NotFoundException(
            "Store not found: " + storeId);
    return new InventoryResource(store);
}
```

Lab 5 — Correct method override (all-or-nothing)

```
// Must redeclare BOTH @GET and @Produces — partial override loses @GET
@Override
@GET
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public List<Product> listAll(String category, int page, int size) {
    return repo.find(category, page, size, true);
}
```

© Training Materials — All Rights Reserved